ECE 153B – Winter 2020
Sensor and Peripheral Interface Design

Lab 3 – UART, SPI, I^2C

Deadline: **February 25, 26, or 27** (at the beginning of your lab section)

Objectives

1. Understand UART
   - Communicating with a computer to print debug messages
   - Interface with the HC-05 bluetooth module
2. Understand the SPI communication protocol
   - Interface with the onboard gyroscope (L3GD20)
3. Understand the I^2C communication protocol
   - Interface with the TC-74 temperature sensor

Grading

Part	Weight
Part A	40%
Part B	30%
Part C	30%

If you have not checked off by the first 30 minutes of the lab section on the deadline date, you will get a 0 for the portions of the lab that you have not finished.

Please note that we take the Honor Code very seriously. You must write all code with only your own team.

Necessary Supplies

- STM32L4 Discovery Board
- Type A Male to Mini B USB Cable
- HC-05 Bluetooth Module
  - USB Bluetooth Receiver (compatible with Windows only) for those of you who need to work on the lab computers because you cannot connect to the HC-05 with your laptop/phone. We tested various devices and found that:
    - iPhones will not connect
    - Androids and Windows laptops (with Bluetooth capabilities) will connect
- TC-74 Temperature Sensor
- Breadboard & Jumper Wires
- 1 kΩ Resistor (x2)

Lab Overview

A. Use UART to control LEDs on the STM32L4 discovery board by sending commands from a terminal on your PC (using Termite). Use UART to interface with a Bluetooth module to control LEDs over a Bluetooth connection.

B. Use SPI to receive gyroscope measurements from the on-board gyroscope. Then, display the information on a terminal on your PC (using Termite).

C. Use I^2C to receive temperature measurements from a temperature sensing module. Then, display the information on the LCD.

Part A – UART

In this part of the lab, you will learn how to set up and use UART (Universal Asynchronous Receiver/Transmitter) to exchange data between the microprocessor and a serial port.

Introduction

The STM32L4 discovery board has 6 embedded UARTs. Table 1 describes the features of each UART that is available on the board. For this lab, we will be using USART2 (for communication through the USB cable) and USART1 (to be used with Bluetooth).

USART modes/features(1)	USART1	USART2	USART3	UART4	UART5	LPUART1
Hardware flow control for modem	X	X	X	X	X	X
Continuous communication using DMA	X	X	X	X	X	X
Multiprocessor communication	X	X	X	X	X	X
Synchronous mode	X	X	X	-	-	-
Smartcard mode	X	X	X	-	-	-
Single-wire half-duplex communication	X	X	X	X	X	X
IrDA SIR ENDEC block	X	X	X	X	X	-
LIN mode	X	X	X	X	X	-
Dual clock domain and wakeup from Stop mode	X	X	X	X	X	X
Receiver timeout interrupt	X	X	X	X	X	-
Modbus communication	X	X	X	X	X	-
Auto baud rate detection	X (4 modes)					-
Driver Enable	X	X	X	X	X	X
LPUART/USART data length	7, 8 and 9 bits					

1. X = supported.

Table 1: STM32L4x6 USART/UART/LPUART Features

Bidirectional communication with UART requires a minimum of two pins: **RX** (for receiving data) and **TX** (for transmitting data). On the STM32L4 discovery board, the TX and RX pins for USART2 are available through the alternative functions of **PD5** and **PD6**, respectively.

Basic Communication with Terminal

First, you will set up UART communication between the STM32L4 discovery board and a terminal on your computer. The goal is to control the LED by sending commands from the terminal. In addition, the board will send back debug messages indicating the LED status.

Use the following steps to help you set up UART and the terminal on your computer.

1. Set up the clock for USART2. You will be modifying the `UART2_Init()` function in `UART.c`. You will be modifying the RCC registers. Refer to section 8.4 in the *STM32L4x6 Reference Manual* for more details about each RCC register and their bits.
   (a) Enable the USART2 clock in the peripheral clock register.
   (b) Select the system clock as the USART2 clock source in the peripherals independent clock configuration register.

2. Configure **PD5** and **PD6** to operate as UART transmitters and receivers. You will be modifying the `UART2_GPIO_Init()` function in `UART.c`. Refer to section 9.4 in the *STM32L4x6 Reference Manual* for more details about each GPIO register and their bits.
   (a) Enable the clock (in RCC) for the GPIO pin.
   (b) Set both GPIO pins to alternative function mode. Refer to the *STM32L476 Pins + Functions* document to find the correct alternative function numbers.
   (c) Set both GPIO pins to very high speed.
   (d) Set both GPIO pins to have a push-pull output type.
   (e) Configure both GPIO pins to use pull-up resistors for I/O.

3. Configure the settings of USART. You will be modifying the `USART_Init()` function in `UART.c`. You will be modifying the USART registers. Note that the argument is `USART_TypeDef* USARTx`, which allows us to define the same configuration that should be applied to the specific USART that we want. Your lines of code should start with `USARTx->[register]`. We will use both USART2 and USART1 in this lab and writing this general code will allow us to configure both in the same way without writing the same code twice – we can simply call `USART_Init(USART2)` or `USART_Init(USART1)`.

   Note that USART must be disabled to modify the settings – ensure that USART is disabled before modifying the registers. Refer to section 36.8 in the *STM32L4x6 Reference Manual* for more details about each USART register and their bits.
   (a) In the control registers, set world length to 8 bits, set oversampling mode to oversampling by 16, and set the number of stop bits to 1.
   (b) Set the baud rate to 9600. To generate the baud rate, you will have to write a value into `USARTx_BRR`.

   Note the word "generate". You should not be writing "9600" into this register. Instead you should write the value that will generate the desired baud rate. The baud rate is computed as follows

   $$\text{TX/RX Baud Rate} = \frac{f_{CLK}}{USARTDIV}$$

   in the case of oversampling by 16. f_{CLK} is the frequency of the system clock and the value $USARTDIV$ is a constant that determines what the baud rate will be. Then, $USARTDIV$ is the value you have to solve for and write into `USARTx_BRR`.
   (c) In the control registers, enable both the transmitter and receiver.
   (d) Now that everything has been set up, enable USART in the control registers.

4. We will be using a program called Termite to communicate with the STM32L4 discovery board. Download the portable version of Termite from this link – this will allow us to run Termite without admin privileges.

   Run `Termite.exe` and you will see a window as shown in Figure 1.

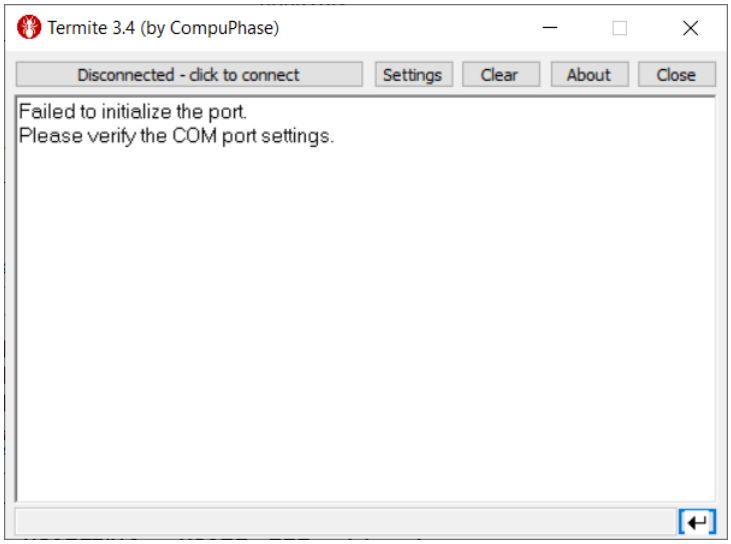

Figure 1

   Click on the **Settings** button and ensure that the **Baud rate**, **Data bits**, and **Stop bits** fields match with the communication parameters that we set in the USART2 registers. See Figure 2.

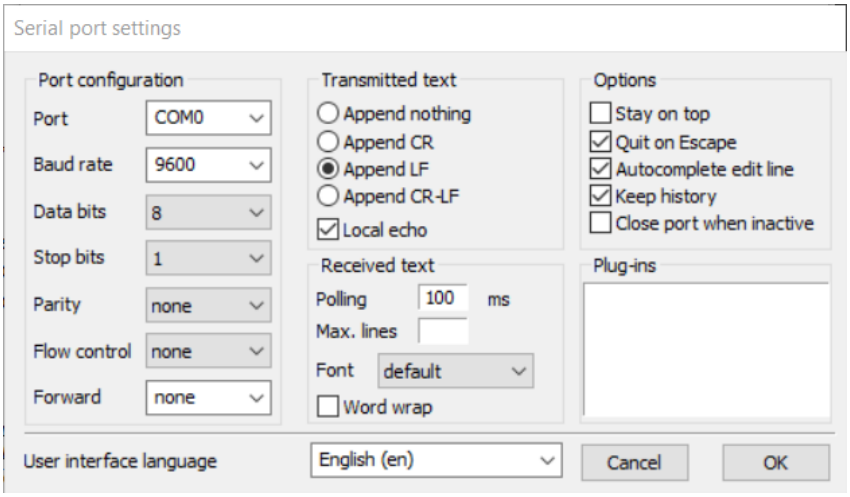

Figure 2

   Exit out of the **Settings** menu and click on the **Disconnected – click to connect** button. When a successful connection is made, you will see the window with a message saying that "Termite is initialized and ready". Note that your COM port may be different. Usually, connecting your STM32L4 discovery board before pressing the **Connect** button will automatically find the necessary COM port. See Figure 3.

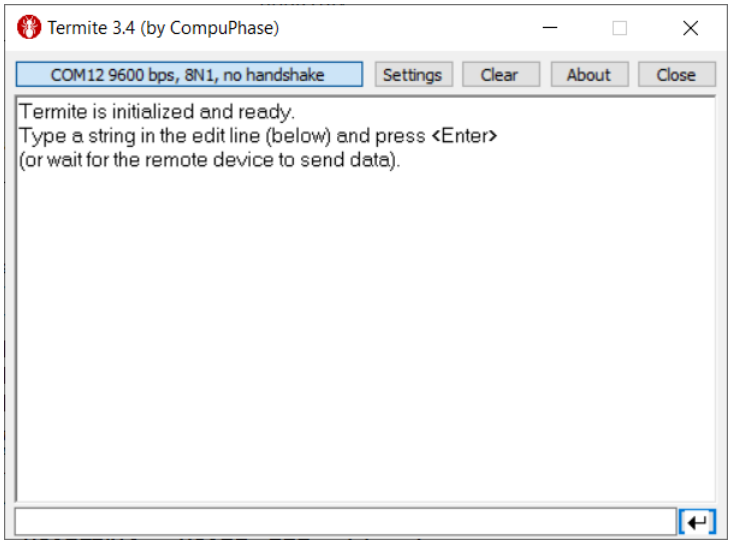

Figure 3

At this point, everything should be set up: UART is configured with the desired communication parameters and Termite is set up as a terminal that will communicate with the STM32L4 discovery board via the UART transmitters/receivers. In the file `UART_printf.c` you will see that we have retargeted the functions `printf()` and `scanf()`. This allows you to:

- Use `printf()` to transmit data to Termite. The data that you send will be printed on the Termite window.
- Use `scanf()` to receive data from Termite. Data can be sent from Termite by typing a message into the terminal and pressing enter.

Using these two functions, your task is to implement the following behavior in the `main()` function. You only need to handle sending one character messages from Termite.

- Send data to the terminal that prompts the user to enter a command. The valid commands are "Y", "y", "N", or "n".
- Receive the response from the terminal.
  - If the response is "Y" or "y", turn the red LED on. In addition, send a message (that may be used for debugging purposes) to the terminal saying that the red LED has been turned on.
  - If the response is "N" or "n", turn the red LED off. In addition, send a message to the terminal saying that the red LED has been turned off.
  - If an unrecognized command is received, send a message to the terminal prompting the user to try again with a valid command.

Interfacing with Bluetooth Module

In this part of the lab, you will set up the HC-05 Bluetooth module and set up UART for communicating with the module. Then, you will connect to the HC-05 Bluetooth module with your phone/laptop or using the lab computers to control the LEDs on the STM32L4 discovery board as you did in the previous part. Refer to the *HC-05 Bluetooth Module Datasheet* for details about the module. Figure 4 shows the connection diagram for the HC-05 Bluetooth module. (Note that the `TX` pin of one device should be connected to the `RX` pin of the other device.)

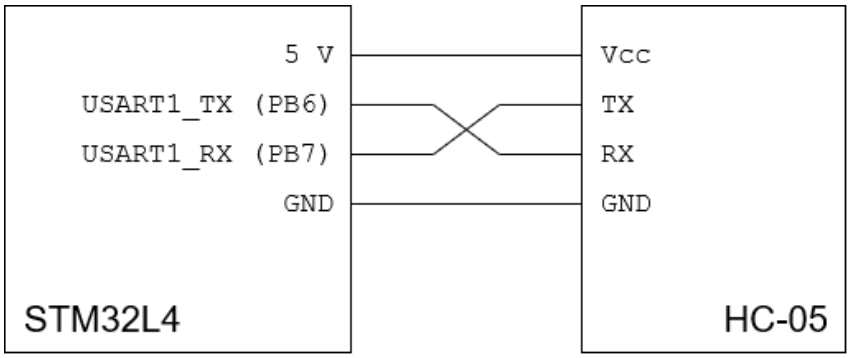

Figure 4: HC-05 Connection Diagram

For this part of the lab, you will be using **PB6** (`USART1_TX`) and **PB7** (`USART1_RX`) for UART communication because we need access to pins that we can connect wires to. Use the following steps to help you set up USART1 and communicate over Bluetooth.

Refer to the steps in the previous part to set up USART1. That is,

1. Modify `UART1_Init()` to set up the USART1 clock.
2. Modify `UART1_GPIO_Init()` to configure **PB6** and **PB7** to operate as UART transmitters and receivers.
3. To send/receive messages using USART1, be sure to go into `UART_printf.c`, comment out the lines in `fputc()` and `fgetc()` corresponding to USART2, and uncomment the lines in `fputc()` and `fgetc()` corresponding to USART1.

Once you have your code running, connect to it using your phone/laptop or the lab computer. The default pairing code is `1234` (which can be changed if you desire by sending the module `AT` commands – refer to the datasheet for more information). The HC-05 also has a red status LED that can help you determine what state it is in. If the LED is blinking rapidly, it is on and is ready to be paired. If the LED is blinking twice approximately every 5 seconds, it is paired to a device.

Establishing Bluetooth Connection with Lab Computers

First, insert the USB Bluetooth receiver into the USB port of the lab computers (it will be recognized as "dongle"). The receiver should be plug-and-play, so there is no additional setup that has to be done on the lap computers. Run your code so that your HC-05 is ready to be paired with another device. In the lab computers, go into **Settings** → **Devices** → **Bluetooth & other devices** and click on the **Add Bluetooth or other device** button (see Figure 5).

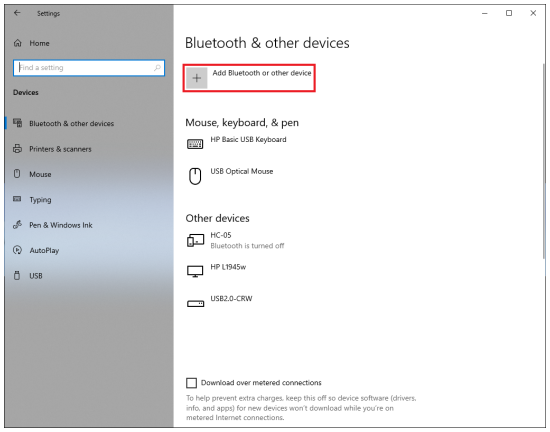

Figure 5

The lab computers do not have Bluetooth capabilities, so the USB Bluetooth receiver allow you to pair with Bluetooth devices. Click on the button for adding Bluetooth devices (see Figure 6) and connect to the HC-05.

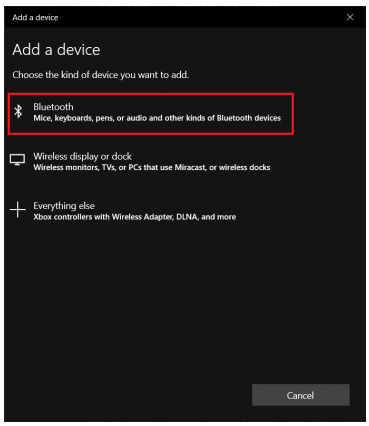

Figure 6

After pairing, you should be able to use Termite as usual for communicating with the STM32L4 board using Bluetooth and UART. You might have to try different COM ports until a connection is established between the lab computer and the HC-05. To know whether you have a connection established, you can check **Settings** → **Devices** → **Bluetooth & other devices** and you will see "Connected" next to HC-05 instead of "Paired" (see Figure 7).

Figure 7

Establishing Bluetooth Connection with Android Phones

Install *Bluetooth Terminal HC-05*. This app provides an easy interface for connecting to nearby Bluetooth devices, connecting with it, and sending messages to the connected Bluetooth device.

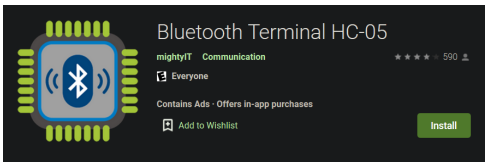

Figure 8

Part B – SPI

In this part of the lab, you will be interfacing with the L3GD20 gyroscope, which is already on the STM32L4 discovery board. You will be communicating with it using SPI. SPI enables communication between *one* master device and multiple slave devices using four wires: the master-in slave-out line (MISO), the master-out slave-in line (MOSI), the serial clock line (SCLK), and the slave select line (SS).

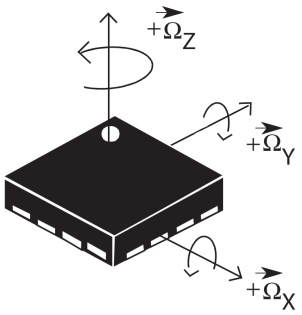

Figure 9

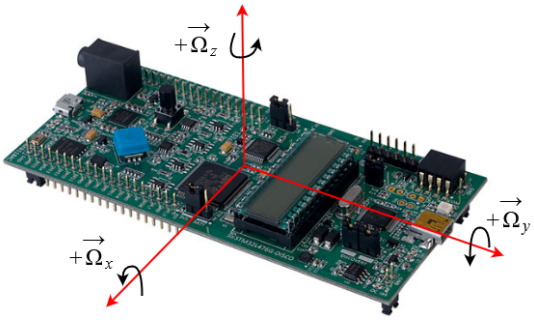

Figure 10

For interfacing with the on-board gyroscope, we will be using SPI2. Set up the GPIO pins **PD1**, **PD3**, and **PD4** for use in SPI communication. Configure these pins to have the settings shown in the table below. You will be writing the GPIO pin initialization code in the `SPI2_GPIO_Init()` function in `SPI.c`.

Pin	Mode	Output Type	Output Speed	Pull-Up/Down
PD1	Alt. Function - `SPI2_SCK`	Push-Pull	Very High	No Pull-Up/Down
PD3	Alt. Function - `SPI2_MISO`	Push-Pull	Very High	No Pull-Up/Down
PD4	Alt. Function - `SPI2_MOSI`	Push-Pull	Very High	No Pull-Up/Down

Configure **PD7** for use as an output pin to be used for chip select. Configure **PD7** to have the settings shown in the table below. You should write this code in the `GYRO_IO_CS_Init()` function in `L3GD20.c`.

Pin	Mode	Output Type	Output Speed	Pull-Up/Down
PD7	Output	Push-Pull	Very High	No Pull-Up/Down

Use the following steps to configure the SPI settings and initialize the gyroscope. You will be modifying the `SPI_Init()` function in `SPI.c`.

1. Set up the clock for SPI in the RCC registers. Refer to section 8.4 of the *STM32L4x6 Reference Manual* for more detailed information about the RCC registers and bits.
   (a) Enable the clock for `SPI2` in the peripheral clock enable register. This code should be written in the `SPI_Init()` function of `SPI.c`.
   (b) Reset `SPI2` by setting the appropriate bit in the peripheral reset register. Afterwards, clear the bits so that `SPI2` does not remain in a reset state.

2. Configure the settings for SPI communication. Similar to when we configured USART settings, ensure that SPI is disabled before modifying the registers. You will only be modifying the SPI control registers. Refer to section 38.6 of the *STM32L4x6 Reference Manual* for more detailed information about the SPI registers and bits. This code should be written in the `SPI_Init()` function of `SPI.c`.
   (a) Configure the serial channel for full duplex communication.
   (b) Configure the communication for 2-line unidirectional data mode.
   (c) Disable the output in bidirectional mode.
   (d) Set the frame format for receiving the MSB first when data is transmitted. In addition, set the data length to 8 bits and set the frame format to be in SPI Motorola mode.
   (e) Set the clock polarity to 0 (the clock will go to 0 when idle). Set the clock phase such that the first clock transition is the first data capture edge.
   (f) Set the baud rate prescaler to 16.
   (g) Disable CRC calculation.
   (h) Set the board to operate in master mode. Enable software slave management and NSS pulse management.
   (i) Set the internal slave select bit.
   (j) Set the FIFO reception threshold to 1/4 (8 bit).
   (k) Enable SPI.
3. Initialize the gyroscope by setting the appropriate bits in its registers. Study the function `GYRO_IO_Write()` as you will be using it to write bits to the gyroscope registers. You should write this code in the `L3GD20_Init()` function in `L3GD20.c`. Refer to section 7 in the *L3GD20 Gyroscope Datasheet* for details about the set of registers in the L3GD20 module. In addition, `L3GD20.h` has many defined macros that may be of use to you.
   (a) Enable all axes and set the gyroscope to normal (active) mode. Set the data rate and bandwidth such that the output data rate is 95 Hz with a cut-off of 25. (You should be writing to control register 1.)
   (b) Select the 2000 dps full scale. (You should be writing to control register 4.)

Write code that will continually get the x-, y-, and z-axis angular velocities from the gyroscope. Study the `GYRO_IO_Read()` function as it is the primary function you will use to get information from the gyroscope.

- Before getting the data from the gyroscope, you need to make sure that the gyroscope has new data available. You must read the gyroscope's status register to verify that there is new data to be read.
- If there is new data to be read, you will get the data by reading the `OUT_<axis>_L` and `OUT_<axis>_H` registers (the measurement for one axis is split into two registers). The `L` stands for "low", indicating the lower bits in the measurement, and the `H` stands for "high", indicating the higher bits in the measurement. You will have to reconstruct the correct measurement from these two segments of data. Note that you will have to convert the reconstructed measurement into dps (degrees per second) – from the datasheet, each unit is 70 mdps. We have defined the structure `L3GD20_Data_t` if you would like to use it to store the axes angular velocity measurements.
- Display the received information on the Termite terminal.

Part C – I^{2}C

In this part of the lab, you will be interfacing with the TC-74 temperature sensor to get and display the temperature data onto the LCD. You are to interface with the temperature sensor using the I^{2}C serial protocol. I^{2}C enables communication between devices using two wires: the **serial data line** (SDA) and the **serial clock line** (SCL).

The TC-74 temperature sensor converts the measured temperature to 8 bits of digital data which can be transmitted out via the I^{2}C interface. As shown in Figure 2, the representable temperature ranges from -65 °C to 127 °C.

Actual Temperature	Registered Temperature	Binary Hex
+130.00°C	+127°C	0111 1111
+127.00°C	+127°C	0111 1111
+126.50°C	+126°C	0111 1110
+25.25°C	+25°C	0001 1001
+0.50°C	0°C	0000 0000
+0.25°C	0°C	0000 0000
0.00°C	0°C	0000 0000
-0.25°C	-1°C	1111 1111
-0.50°C	-1°C	1111 1111
-0.75°C	-1°C	1111 1111
-1.00°C	-1°C	1111 1111
-25.00°C	-25°C	1110 0111
-25.25°C	-26°C	1110 0110
-54.75°C	-55°C	1100 1001
-55.00°C	-55°C	1100 1001
-65.00°C	-65°C	1011 1111

Table 2: Temperature to Digital Value Conversion

The temperature resolution is 1 °C and the conversion rate is 8 samples per second. Multiple sensors can be connected to the same I^{2}C bus and can communicate with each other as long as they all have unique addresses. For this lab, you will only have to interface with one temperature sensor. The temperature sensors can have one of eight different addresses from `1001000` to `1001111`. Table 3 shows the address of the temperature sensor that corresponds to the part number TC74Ax.

Part Number	Default Address
A0	1001000
A1	1001001
A2	1001010
A3	1001011
A4	1001100
A5	1001101
A6	1001110
A7	1001111

Table 3: Address Corresponding to Part Number

Figure 11 shows the connection diagram for the temperature sensor.

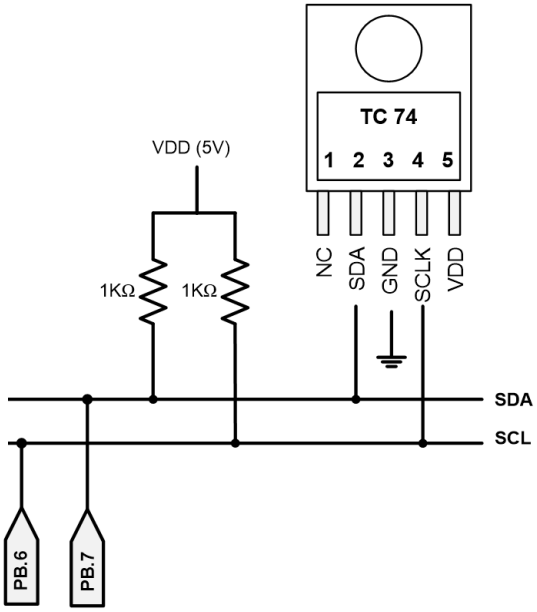

Figure 11: Temperature Sensor Connection Diagram

You will use **PB6** and **PB7** for `I2C1_SCL` and `I2C1_SDA`, respectively. Configure **PB6** and **PB7** with the following settings. Write your code in the `I2C_GPIO_Init()` function in `I2C.c`.

Pin	Mode	Output Type	Output Speed	Pull-Up/Down
PB6	Alt. Function - `I2C1_SCL`	Open-Drain	Very High	Pull-Up
PB7	Alt. Function - `I2C1_SDA`	Open-Drain	Very High	Pull-Up

Use the following steps to configure the I^2C settings. Write your code in the `I2C_Initialization()` function in `I2C.c`.

1. Set up the clock for I^2C in the RCC registers. Refer to section 8.4 of the *STM32L4x6 Reference Manual* for detailed information about the RCC registers and their bits.
   (a) Enable the clock for `I2C1` in the peripheral clock enable register.
   (b) Set the system clock as the clock source for `I2C1` in the peripherals independent clock configuration register.
   (c) Reset `I2C1` by setting bits in the peripheral reset register. After doing so, clear the bits so that `I2C1` does not remain in a reset state.
2. Configure the I^2C registers for communication with the temperature sensor. Similar to when we configured USART settings, ensure that I^2C is disabled before modifying the registers. Refer to section 35.7 of the *STM32L4x6 Reference Manual* for more information about the I^2C registers and their bits.
   (a) Enable the analog noise filter, disable the digital noise filter, enable error interrupts, and enable clock stretching. Set the master to operate in 7-bit addressing mode. Enable automatic end mode and `NACK` generation. (These settings are all in the control registers).

(b) Set the values in the timing register. This guarantees correct data hold and setup times that are used in master/slave modes. The timing register stores several values:

- `PRESC` – This value is the timing prescaler. It is used to generate the clock period that will be used for the data setup, data hold, and SCL high/low counters.

$$f_{\text{PRESC}} = \frac{f_{\text{I2CCLK}}}{\texttt{PRESC} + 1} \qquad \Longleftrightarrow \qquad t_{\text{PRESC}} = (\texttt{PRESC} + 1) \times t_{\text{I2CCLK}}$$

- `SCLDEL` – This value determines the minimum data setup time, which is a delay between an SDA edge and the next SCL rising edge in transmission mode. The generated delay time is

$$t_{\text{SCLDEL}} = (\texttt{SCLDEL} + 1) \times t_{\text{PRESC}}$$

- `SDADEL` – This value determines the minimum data hold time, which is a delay between the SCL falling edge and the next SDA edge in transmission mode. The generated delay time is

$$t_{\text{SDADEL}} = (\texttt{SDADEL} + 1) \times t_{\text{PRESC}}$$

- `SCLH` – This value determines the minimum period for the high phase of the clock signal. The generated high period is

$$t_{\text{SCLH}} = (\texttt{SCLH} + 1) \times t_{\text{PRESC}}$$

- `SCLL` – This value determines the minimum period for the low phase of the clock signal. The generated low period is

$$t_{\text{SCLL}} = (\texttt{SCLL} + 1) \times t_{\text{PRESC}}$$

Figure 12 shows an illustration of the data hold and data setup times.

These values must meet the requirements of the TC-74 temperature sensor. Keep in mind that the system clock (which we configured to be the source of the I^2C clock) has a frequency of 80 MHz. First, write down the prescaler value that you will use. Then, look in *Digital Thermal Sensor - TC74 - Data Sheet* to find the specifications for these timings. Using this information, you can calculate what values need to be written to the timing register such that the timing guarantees meet the temperature sensor's specifications. Macros for the values' position in the timing register have been defined for you.

`PRESC` = _____

Min. Low Clock Period = _____

Min. High Clock Period = _____

Min. Data Setup Time = _____

Min. Data Hold Time = _____

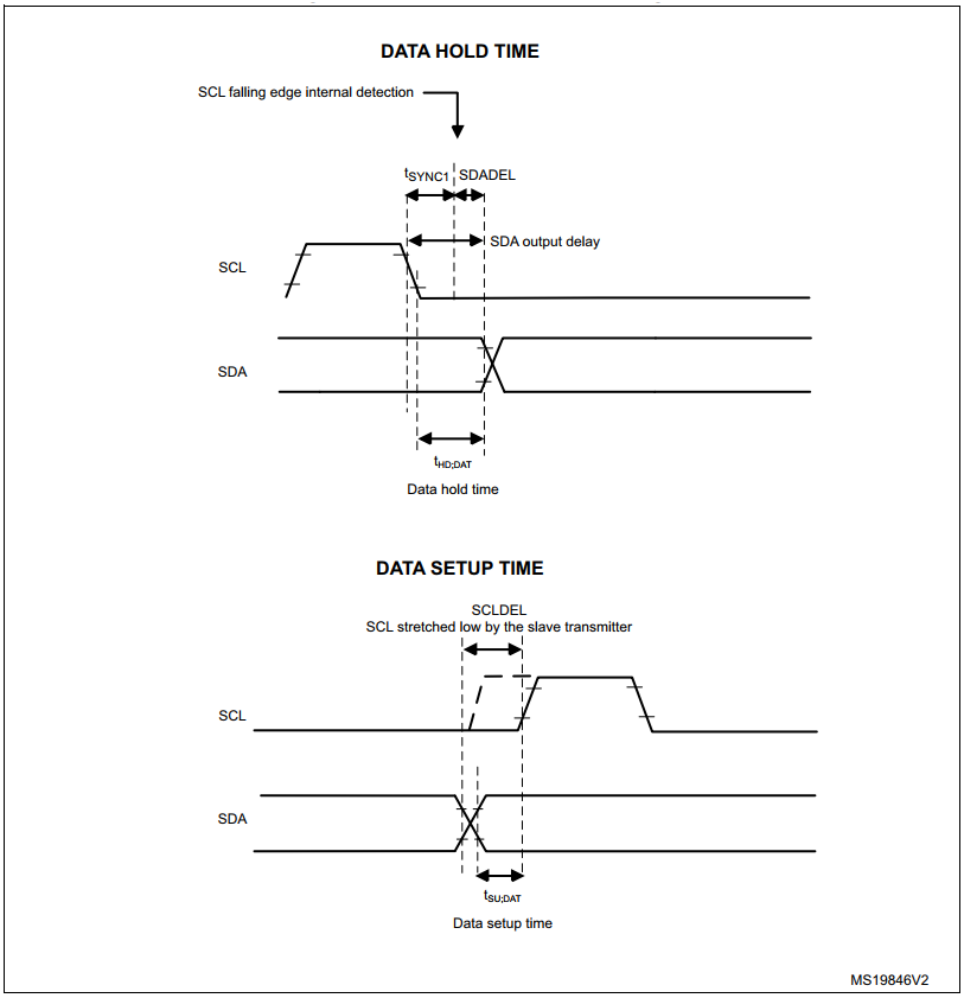

Figure 12: Data Hold and Setup Timimg

3. Set your own address in the own address registers. To modify the address, you must first disable the own address. Do this for only Own Address 1 – we do not need Own Address 2 (ensure that it remains disabled).
   (a) Set own address to 7-bit mode.
   (b) Write the own address that you want to use – you are free to use `OwnAddr = 0x52` that is provided in the code.
   (c) Enable own address.
4. Enable I^2C in the control register.

In `I2C.c`, study the functions `I2C_SendData()` and `I2C_ReceiveData()`. These are the two functions that you will use to communicate with the temperature sensor. In *Digital Thermal Sensor - TC74 - Data Sheet*, find the command byte that you must send to the temperature sensor to get a temperature measurement.

Read Temperature Command Byte = `0x`_____

Write code in `main()` that will constantly get temperature measurements from the sensor. Furthermore, display the received measurement onto the LCD (just the number is sufficient).

References

[1] Yifeng Zhu, "Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C", ISBN: 0982692633

[2] STM32L47x6 Advanced ARM-based 32-bit MCUs Reference Manual

[3] Discovery Kit with STM32L476VG MCU User Manual

[4] HC-05 Bluetooth Module Datasheet

[5] L3GD20 Gyroscope Datasheet

[6] TC-74 Temperature Sensor Datasheet